

SDEV2171

Lesson 13: Loading, Errors, and Race Conditions

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, you should be able to:

- Show the user whether a request is loading, has succeeded, or has failed
- Model a request with one status value instead of separate flags
- Explain how `fetch` reacts to HTTP errors and network errors, and handle both
- Cancel out-of-date requests with `AbortController`
- Refactor request handling into a custom hook any component can reuse
- Describe how production apps manage data from a server

WE ARE ESSENTIAL TO ALBERTA | NAIT

Today

Time	Activity
0:00	Run the backend and the starter app
0:10	What the starter app gets wrong
0:15	Showing request status
0:30	Handling failed requests
0:45	Break
0:55	Retrying a request
1:02	Out-of-order responses
1:20	Refactoring into a custom hook
1:32	How production apps do this
1:37	Challenges and checkpoint

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Lesson Backend

In its own terminal, run `node server.js` from the `backend` folder, and leave it running all class. It serves study steps at `http://localhost:3000/api/study-steps`. Add these query parameters to control how it answers:

Parameter	What the server does
`q=che`	returns only steps whose label contains "che"
`delay=1500`	waits 1500 ms before answering
`error=1`	answers with a 500 and `{ "error": "..." }`
`race=1`	answers shorter searches more slowly
`flaky=2`	fails twice for the same URL, then succeeds

Its terminal logs every request, and logs `cancelled by the app` when the app cancels one.

WE ARE ESSENTIAL TO ALBERTA | NAIT

localhost Depends on Where the App Runs

`localhost` means the machine the code is running on. On an emulator or a phone, that isn't your computer.

App running in	Use this host for your computer
Browser	`localhost`
iOS simulator	`localhost`
Android emulator	`10.0.2.2`
Physical phone	your computer's LAN IP, on the same Wi-Fi

`src/lib/apiBaseUrl.js` picks the host for you, based on `Platform.OS`. On a phone, set it to your LAN IP yourself.

Development builds allow plain `http`. Release builds block it by default, so a production API has to use `https`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

What the Starter App Shows

Try each situation in the starter app. Almost all of them show the same message:

Situation	The screen shows	It should show
The request is still running	"No study steps yet."	a loading indicator
The server answered with a 500	"No study steps yet."	the error, and a Retry button
The server can’t be reached	"No study steps yet."	the error, and a Retry button
The search matched nothing	"No study steps yet."	that nothing matched the search
You typed quickly	results for an older search	results for what’s in the box

A user can’t tell a slow network from a broken app. Each of these situations needs its own UI.

WE ARE ESSENTIAL TO ALBERTA | NAIT

One Status Instead of Flags

Separate flags let the screen get into combinations that mean nothing:

```
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);
// Both can be set at once. What should the screen show then?
```

A single status value can only be one thing at a time:

```
const [status, setStatus] = useState('loading'); // 'loading' | 'success' | 'error'
```

Set `status` exactly once on each path through the request. There's no flag left to reset, so you don't need `finally`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

How to Build and Commit a Change

Build every change in this order:

1. Add the state, with a hard-coded starting value
2. Write every UI branch that state can produce
3. Check each branch by changing the hard-coded value and saving
4. Wire the logic that sets the state
5. Test it against the backend
6. Commit

Building the UI first means a screen that looks wrong is a UI bug, not a network bug. Wiring the logic last means you only debug one thing at a time.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Showing Request Status

Add a status state, and render one branch for each value:

`status`	**Render**
`'loading'`	an `ActivityIndicator` and "Loading study steps…"
`'error'`	"Couldn’t load study steps."
`'success'` with no results	No study steps match "`query`".
`'success'` with results	the list

Then set `status` in the request: `'loading'` before `fetch`, `'success'` after storing the data, and `'error'` in `catch`.

Test it with `delay=1500` and with `delay=1500&error=1`, and search for `zzz`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

HTTP Status Codes: The Server's Answer

Every HTTP response carries a status code. The server uses it to say whether it did what you asked:

Code	Meaning	From our backend
2xx	success	`200` with the list of study steps
4xx	the request was wrong	`404` for `/api/study-stepz`
5xx	the server failed	`500` for `error=1`

A failed request still gets a response. Most APIs also put an explanation in the body. Ours sends `{ "error": "The server failed on purpose (error=1)." }`.

Open `http://localhost:3000/api/study-steps?error=1` and `http://localhost:3000/api/study-stepz` in a browser to see both.

WE ARE ESSENTIAL TO ALBERTA | NAIT

How fetch Reacts to Each Outcome

`fetch` only cares whether an answer arrived. It doesn't judge the status code, so checking it is your code's job:

What happened	What `fetch` does	What our code does
The server answered 2xx	resolves, `response.ok` is `true`	reads the JSON and shows the list
The server answered 4xx or 5xx	resolves, `response.ok` is `false`	reads the body's `error` and throws it
No answer: server down, wrong host, offline	rejects with a `TypeError`	shows the error in `catch`
We cancelled the request	rejects with an `AbortError`	ignores it (later today)

WE ARE ESSENTIAL TO ALBERTA | NAIT

Showing the Error Message

Store the error message in state, and when `response.ok` is `false`, throw the server's message:

```
const [errorMessage, setErrorMessage] = useState('');
```

```
if (!response.ok) {
  const body = await response.json().catch(() => ({}));
  throw new Error(body.error ?? `Server returned ${response.status}`);
}
```

In `catch`, save it with `setErrorMessage(error.message)`, and show it in the error card. Both kinds of failure now reach the same `catch`. Test both:

- With `error=1`, the screen should show "The server failed on purpose (error=1)."
- With the server stopped, it should show `Failed to fetch` in the browser, or `Network request failed` on a device.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Break

10 minutes

WE ARE ESSENTIAL TO ALBERTA
NAIT

Retrying a Request

The request function is defined inside the effect, so a button can’t call it. Give the effect a counter to depend on, and have Retry change it:

```
const [reloadKey, setReloadKey] = useState(0);
```

```
}, [query, reloadKey]);
```

```
<Pressable style={styles.button} onPress={() => setReloadKey((key) => key + 1)}>
  <Text style={styles.buttonLabel}>Retry</Text>
</Pressable>
```

Test it: stop the server, open the screen, start the server again, and press Retry.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Responses Arrive Out of Order

Set `TEST_PARAMS` to `'race=1'` and type `che` with about 150 ms between keys:

Request	Sent at	Answered at
`q=c`	0 ms	1600 ms
`q=ch`	150 ms	1350 ms
`q=che`	300 ms	1100 ms

Every response calls `setSteps`, so the last one to arrive wins. Here that's the response for `c`: the screen shows 9 results while the box says `che`.

Real networks reorder responses too. `race=1` makes it happen every time, so you can test it.

WE ARE ESSENTIAL TO ALBERTA | NAIT

AbortController: A Bomb on the Request

Think of an `AbortController` as a bomb you can attach to a request:

- `new AbortController()` builds the bomb
- `controller.signal` is the wire. Pass it to `fetch` to attach the bomb to that request
- `controller.abort()` sets it off. The connection is dropped, and the `fetch` promise rejects with an `AbortError`

```
const controller = new AbortController();
fetch(url, { signal: controller.signal });
controller.abort(); // the request stops, and fetch rejects
```

A bomb only goes off once. After `abort()`, the controller stays aborted, so every request needs its own.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Cancelling Fixes the Race

Each run of the effect builds its own bomb and attaches it to its own request. Before React runs the effect again, it runs the previous run's cleanup, and the cleanup sets off the old bomb.

You type	What React does
`c`	runs the effect: a request for `c`, with bomb 1
`h`	cleanup sets off bomb 1, then the effect runs: a request for `ch`, with bomb 2
`e`	cleanup sets off bomb 2, then the effect runs: a request for `che`, with bomb 3

By the time the slow answer for `c` would arrive, its request is gone. Only the newest request can reach `setSteps`.

The cancelled requests still reject, so they land in `catch`. Check `controller.signal.aborted` first, and return without touching state.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Cancelling in the Effect

Add the four pieces in this order:

```
useEffect(() => {
  const controller = new AbortController();            // 1. build the bomb
  async function loadSteps() {
    try {
      const response = await fetch(url, { signal: controller.signal }); // 2. attach it
      // ...
    } catch (error) {
      if (controller.signal.aborted) return;            // 4. ignore our own cancels
      // ...
    }
  }
  loadSteps();
  return () => controller.abort();                      // 3. set it off in cleanup
}, [query, reloadKey]);
```

Test it with `race=1`: typing `che` quickly should show 3 results.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Moving Request Logic Into a Hook

The screen now does two jobs. It manages a request (status, error message, retry, cancelling), and it decides what to show. The request half has nothing to do with study steps, and every screen that fetches data would need a copy of it.

Move that half into a custom hook, `useFetch(url)`. The screen keeps only what's specific to it: building the URL, and rendering each status.

A custom hook is a function whose name starts with `use` and that calls other hooks. Every component that calls it gets its own separate copy of the hook's state.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Define the Hook’s Contract First

Before you move any code, decide what the hook takes and what it returns. Write that with hard-coded values in `src/hooks/useFetch.js`:

```
export function useFetch(url) {
  // TODO: move the request state and the effect in here
  return { status: 'success', data: [], errorMessage: '', retry: () => {} };
}
```

Then cut the state and the effect out of the screen, and call the hook instead:

```
const url = `${API_BASE_URL}/study-steps?q=${encodeURIComponent(query)}&${TEST_PARAMS}`;
const { status, data, errorMessage, retry } = useFetch(url);
const steps = data ?? [];
```

Save. The screen shows the no-matches message, which proves the screen and the hook are connected before any logic moves.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Filling In the Hook

Paste the state and the effect into the hook, and make three changes:

1. Rename `steps` to `data`, starting as `null`. The hook doesn't know what it's fetching
2. Depend on `url` instead of `query`. The URL already contains the query
3. Return a `retry` function, so the screen doesn't need to know `reloadKey` exists

```
  }, [url, reloadKey]);

  function retry() {
    setReloadKey((key) => key + 1);
  }

  return { status, data, errorMessage, retry };
}
```

In the screen, change Retry to `onPress={retry}`, then rerun every earlier test.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Reusing the Hook

Show the number of study steps on the Home screen. It needs the same loading and error handling, and now that takes one line:

```
const { status, data } = useFetch(`${API_BASE_URL}/study-steps`);
```

```
<Text style={styles.body}>
  {status === 'loading' && 'Loading study steps...'}
  {status === 'error' && "Couldn't load study steps."}
  {status === 'success' && `${data.length} study steps saved`}
</Text>
```

Home and Details now make two independent requests. Each call to `useFetch` has its own state, its own controller, and its own `retry`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Server Data Is a Cache

Data from a server isn't really your component's state. It's a copy of something that lives on the server, and it can go out of date as soon as it arrives. Production apps treat it as a cache:

- Results are stored by a key, such as `['study-steps', 'che']`. Two screens that ask for the same key share one request and one result
- Going back to a screen shows the cached data immediately, then refetches in the background and updates it. This pattern is called stale-while-revalidate
- Data is refetched when it's likely to have changed: when the app comes back to the foreground, or when the network reconnects

Libraries that do this: TanStack Query (the most common in React and React Native), SWR, RTK Query (for apps that use Redux), and Apollo Client (for GraphQL APIs).

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Same Screen With TanStack Query

```
const url = `${API_BASE_URL}/study-steps?q=${encodeURIComponent(query)}`;
const { status, data, error, refetch } = useQuery({
  queryKey: ['study-steps', query],
  queryFn: ({ signal }) => fetchJson(url, { signal }),
});
```

`fetchJson` is your `fetch` plus the `response.ok` check. The rest maps onto today's code:

Today	TanStack Query
`status` of `'loading'`	`status` of `'pending'`
`retry()`	`refetch()`
`[url, reloadKey]` dependencies	`queryKey`
your `AbortController`	the `signal` it passes in, aborted when out of date

WE ARE ESSENTIAL TO ALBERTA | NAIT

What a Data Library Adds

Beyond what you built today, TanStack Query's defaults:

- retry a failed query 3 times, waiting 1 s, 2 s, then 4 s, before showing an error
- keep results in the cache for 5 minutes after no screen is using them
- treat cached data as stale right away, and refetch it in the background when a screen mounts, the app regains focus, or the network reconnects
- with `placeholderData: keepPreviousData`, keep showing the old list while a new search loads, instead of a spinner on every keystroke

React Native has no browser focus or online events, so you connect them yourself: `focusManager` to React Native's `AppState`, and `onlineManager` to `@react-native-community/netinfo` or `expo-network`. The TanStack Query docs have a React Native page with the code.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenges

Pick one of these challenges to finish after class. Each slide explains the problem briefly. Look up the term it names, read until you understand it, then implement a solution.

Each challenge gives you:

- the `TEST_PARAMS` value, or the setup, that makes the problem happen
- what your solution has to do
- a hint, and a starter snippet

Build it in `useFetch` or alongside it, and commit it on its own, following the build order. The README has a full solution for each one in a collapsed section. Try it yourself before you look.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Debounce the Search

Every keystroke sends a request. Typing `check` sends five, and four of them are cancelled right away. Look up **debounce**.

Trigger it: set `TEST_PARAMS` to `'race=1'`, type `check`, and count the requests in the server terminal.

Your solution: send a request only after the user has stopped typing for 300 ms. Typing `check` quickly should log one request.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Debounce the Search: Hint

Hint: delay the value, not the request. Write a hook that returns the query only after it stops changing, and build the URL from that.

```
export function useDebouncedValue(value, delayMs) {
  const [debouncedValue, setDebouncedValue] = useState(value);
  useEffect(() => {
    // TODO: after delayMs, copy value into debouncedValue
    // TODO: return a cleanup that cancels the pending timer
  }, [value, delayMs]);
  return debouncedValue;
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Time Out Slow Requests

`fetch` has no time limit. If a server accepts the connection but never answers, the spinner can stay up for minutes. Look up **fetch timeout with AbortController**.

Trigger it: set `TEST_PARAMS` to `'delay=8000'`.

Your solution: give up after 5 seconds, and show "The server took more than 5 seconds to answer."

WE ARE ESSENTIAL TO ALBERTA | NAIT

Time Out Slow Requests: Hint

Hint: a timer can set off the same bomb the cleanup uses. Keep a flag, so `catch` can tell a timeout from a cleanup.

```
const controller = new AbortController();
let timedOut = false;
const timeoutId = setTimeout(() => {
  // TODO: mark it as timed out, then abort
}, 5000);
// In catch: TODO return early only if this wasn't a timeout
// In the cleanup: TODO clear the timer as well as aborting
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Retry Automatically With Backoff

Many failures are temporary: the network drops for a second, or the server is restarting. Production apps retry a few times before showing an error, and wait longer each time so they don't flood a server that's struggling. Look up **exponential backoff**.

Trigger it: set `TEST_PARAMS` to `'flaky=2'`, then to `'flaky=5'`.

Your solution: retry up to 3 times, waiting 500 ms, 1000 ms, then 2000 ms. With `flaky=2`, the list appears after the third request with no error. With `flaky=5`, the error appears after the fourth.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Retry Automatically With Backoff: Hint

Hint: put the request in a loop, and wait between attempts. Check `controller.signal.aborted` after every wait.

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

for (let attempt = 0; attempt <= 3; attempt++) {
  try {
    // TODO: the request. On success, set data and status, then return
  } catch (error) {
    // TODO: stop if aborted. Set the error if this was the last attempt
    await wait(500 * 2 ** attempt);
  }
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Readable Network Errors

`Failed to fetch` and `Network request failed` mean nothing to a user, and the wording is different on every platform. Look up **fetch TypeError**.

Trigger it: stop the server. Then also try a wrong port in `src/lib/apiBaseUrl.js`.

Your solution: when no response arrived, show "Can’t reach the server. Check your connection and try again." Keep showing the server’s own message for HTTP errors.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Readable Network Errors: Hint

Hint: `fetch` rejects with a `TypeError` only when no response arrived. Every HTTP error reaches `catch` as the `Error` you threw yourself.

```
} catch (error) {
  if (controller.signal.aborted) return;
  // TODO: if error is a TypeError, use the readable message instead
  setErrorMessage(error.message);
  setStatus('error');
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

- What does your screen show for a 500, and for a stopped server? Why does the code reach each one differently?
- What can `isLoading` plus `error` represent that `status` can't?
- In the race test, which request answered last, and why did it overwrite the others?
- When does an effect's cleanup function run, and what does ours do?
- What does `useFetch` return, and why doesn't the screen need `reloadKey` anymore?

WE ARE ESSENTIAL TO ALBERTA | NAIT

Before You Leave

1. Check that your screen handles loading, success, server errors, network errors, no matches, and fast typing
2. Set `TEST_PARAMS` back to `'delay=1500'`
3. Commit and push your work
4. Record your status: **resolved**, **partial**, or **blocked**
5. Pick a challenge, and do the exercise at the end of the lesson README

WE ARE ESSENTIAL TO ALBERTA | NAIT